MALLA REDDY TECHNICAL CAMPUS
A Constituent Unit of MALLA REDDY VISHWAVIDYAPEETH
Deemed to be University
Approved by UGC & AICTE New Delhi
Maisammaguda, Medchal (Dist), Hyderabad - 500100, Telangana

PUBLIC FACILITY FEEDBACK

APEX
backend enigneering
and system challenge hackathon

Presented by :
Niharika
Hasini
Vaishnavi
Nagalakshmi

PROBLEM STATEMENT LACK OF DIRECT REPORTING:

Citizens frequently encounter damaged or unhygienic public amenities (parks, libraries, restrooms, transit stops) with no quick digital mechanism to report them.Delayed Maintenance: Municipal authorities lack real-time visibility into local infrastructure conditions, slowing down issue resolution.Absence of Accountability: Facility tracking and citizen feedback remain paper-based, fragmented, or entirely untracked across municipal departments.

PROPOSED SOLUTIONCENTRALIZED WEB PLATFORM:

A unified single-page application connecting citizens directly to local facility records.Community-Driven Transparency: Publicly visible ratings and comments that keep municipal infrastructure accountable.Role-Based Access Control: Dual-interface model allowing citizens to browse and review, while municipal administrators manage and update facilities.Instant Record Synchronization: Real-time persistence and retrieval powered by an asynchronous REST API.

SYSTEM ARCHITECTURE & DATA FLOWFRONTEND LAYER (CLIENT):

React.js (Vite) running a reactive UI, managing client states with React Hooks (useState, useEffect), and making asynchronous HTTP calls via the Fetch API.Backend Layer (API Engine): Node.js and Express.js handling REST routing, payload parsing, CORS management, and custom authentication middleware.Data Persistence Layer: MongoDB database managed through Mongoose Object Data Modeling (ODM) to guarantee data integrity.End-to-End Data Flow: Client Interface $\leftrightarrow$ REST Endpoints (JSON) $\leftrightarrow$ Express Middleware $\leftrightarrow$ MongoDB Collections.

TECHNOLOGY STACK & MODULES USEDREACT.JS & VITE:

Powers the single-page application interface with fast local compilation and real-time DOM rendering.Node.js & Express.js: Event-driven, non-blocking runtime environment executing REST API routing and controllers.MongoDB & Mongoose: NoSQL document database utilizing defined schemas, types, and document cross-referencing.bcryptjs: Salted one-way cryptographic hashing for secure password storage in the database.jsonwebtoken (JWT): Stateless token generation and signature verification for authenticated session access.cors: Configured to securely permit cross-origin requests between the React frontend and the Express API server.

DATABASE SCHEMA & ENTITY MODELINGUSER SCHEMA:

Fields: name, email (unique index), password (hashed string), role (enum: 'user' | 'admin').Facility Schema:Fields: name, category (Park, Hospital, Restroom, Library, Bus Stop), address, city.Feedback Schema:Fields: facilityId (MongoDB ObjectId foreign reference linking to Facility), userName, rating (1 to 5 numeric value), comment, status ('Pending' | 'Resolved')

REST API DESIGN & CRUD IMPLEMENTATIONCREATE (POST):

/api/register – Creates a new user with an encrypted password./api/facilities – Adds a new public facility record (Admin privilege)./api/feedback – Submits a rating and comment (Protected route requiring Bearer JWT).Read (GET):/api/facilities – Retrieves the full catalog of public facilities./api/feedback/:facilityId – Retrieves all user reviews specific to a facility ID.Update (PUT):/api/feedback/:id/status – Updates feedback resolution state (e.g., Pending to Resolved).Delete (DELETE):/api/facilities/:id – Deletes decommissioned public facilities (Admin privilege).

SECURITY & ROLE-BASED ACCESS CONTROL (RBAC)CRYPTOGRAPHIC STORAGE:

Passwords are never stored in plain text; bcryptjs salts and hashes strings before saving them to MongoDB.Token-Based Route Protection: Custom Express middleware (verifyToken) checks the HTTP Authorization header, extracts the Bearer token, verifies the signature, and rejects unauthorized requests with HTTP 401/403.Conditional Frontend Rendering: Administrative UI components (like the "Add Facility" form and delete actions) are rendered dynamically based on the verified role payload of the logged-in user.

RESULTS & PROJECT DEMONSTRATIONFULL-STACK EXECUTION:

Both services operating concurrently (Node/Express API on port 5000 and Vite/React on port 5173).Interactive Dashboard: Citizens can filter facilities, view ratings, and publish immediate reviews.Live Administrative Management: Verified ability to add facilities, inspect incoming complaints, and delete outdated entries in real time.Strict Error Handling: API responses mapped to standard HTTP status codes (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden).

FUTURE SCOPE & CONCLUSIONGEOSPATIAL PROXIMITY QUERIES:

Implementing MongoDB 2dsphere indexes to locate nearby facilities using browser GPS coordinates.Damage Image Uploads: Integrating cloud storage (Cloudinary / AWS S3) via Multer so citizens can upload photo proof of maintenance issues.Automated Rating Engines: Running MongoDB aggregation pipelines to automatically compute rolling average facility scores upon each submission.Conclusion: The project demonstrates a working, full-stack MERN application delivering practical civic utility with secure data flow and complete CRUD functionality.

THANK YOU !